

Light-SABRE: A lightweight encrypted data distribution system for Internet of Things

Nakul Ghatge¹[0000-0001-5811-4789], Shohei Mitani¹[0009-0002-4954-8061] and Hirofumi Ueda¹[0009-0008-4873-7534]

¹ NEC Corporation, Japan

Abstract. The ubiquitous and pervasive nature of Internet of Things (IoT) has raised concerns over storage and transmission of sensitive personal data. Therefore, developing technologies to preserve privacy during data distribution in IoT is paramount for security researchers. Until now, the data owners have had little to no control over their private data collected by the IoT devices, which is stored with proprietary cloud service providers and possibly shared among third parties without the consent of the data owners. The sorry state of data owners arises primarily because the privacy policies of the storage and sharing service providers are not fully transparent as well as difficult to understand. Moreover, the service providers are often than not, keen to analyze and share personal information for their own benefit. As a solution to the privacy problem, this paper proposes a novel approach to combine privacy preserving sticky policy approach with the Attribute-Based Encryption (ABE) technique with the goal of enhancing data control for owners and allowing fine-grained sharing of data. The technique described in this paper utilizes Proxy Re-Encryption (PRE) of transmitted data which prevents even the data storage server from being able to decrypt it. We present our Light-SABRE (**L**ightweight **S**ticky **A**tribute-**B**ased proxy **R**e-**E**ncryption) architecture which effectively combines the above-mentioned approaches to provide a platform for a private data distribution system for IoT devices.

Keywords: Internet of Things, Privacy, Sticky Policies, Attribute-Based Encryption, Proxy Re-Encryption.

1 Introduction

The rapid development and deployment of low-powered smart devices has led to increased pervasiveness of IoT ecosystem in sensitive and personal areas of life such as healthcare, military, finance and transportation [1]. These devices have limited computational power and are designed to perform only the specific tasks; hence they are not inherently secure [2]. Low computational power implies that they cannot use security functions which require large computational power such as cryptographic mechanisms, rendering them easy to leak sensitive information [3].

On the other hand, the storage and distribution of data through these smart devices in IoT ecosystem is governed by the proprietary service providers, usually hosted on

cloud and who apply their own privacy and access control policies on the data. These cloud service providers (or CSPs) may often use sensitive personal data for improving services, analytics and statistics, or selling it to third parties for profits [4]. On one hand, it may generate some value to the users, but on the other hand it causes serious privacy implications to the data owners due to possible abuse and leakage of sensitive data [5]. Data owners rarely have any control over privacy and security of their personal data during data distribution due to ambiguous “terms and conditions”, and inability to set their own policies [6].

In this direction, sticky policies [7] is a promising approach to preserve privacy and control for data owners over their personal data. The sticky policies establish conditions and constraints over data access and use through its entire life cycle, making it feasible for the data owners to freely set their own access control policies over their private data. More specifically, in the reference model of sticky policies [8], the data owners send encrypted data along with its access policy to the service providers. The service providers then contact the trust authority to obtain the access permissions on the received data.

However, ensuring data privacy through sticky policies in IoT scenario is challenging. In sensitive areas of application such as healthcare and finance, it is necessary to ensure that not even the cloud service provider shall be able to decrypt personal data before distributing to the authorized third party to preserve data privacy. In the sticky policy model [8] proposed by Sicari et al, the cloud service providers may request the decryption keys from the trusted authorities (TAs) and thus can access sensitive user data. In a more private world, the service providers cannot be trusted.

To ensure privacy even from the cloud service providers, cryptographic techniques such as Proxy Re-Encryption (PRE) have been greatly studied in the past [9, 10]. PRE can convert a ciphertext encrypted (and/or decrypted) by one key to the ciphertext which can be decrypted by another key without revealing the plaintext information. By allowing the service providers to only perform the PRE, the trust on the service providers can be reduced and privacy from them can be achieved during data sharing.

In certain areas of application such as healthcare, universities, industrial control systems and operational technologies etc., fine-grained sharing and access control of data is desirable [11]. Fine grained data access control allows for flexible and dynamic permissions on user data with the help of a set policy. To achieve that, the Attribute-Based Encryption (ABE) has been studied quite thoroughly in the last two decades [12, 13]. To provide an overview, the ABE operates over access policies defined over attribute conditions and constraints and using them it converts plaintext data into a ciphertext such that only the parties which present attributes such that they fully satisfy a specified policy shall be able to decrypt the ciphertext. It comes in two forms, Key-Policy Attribute-Based Encryption (KP-ABE) where the decryption keys are generated from the policy, while the encryption keys are generated from the attributes [14], and Ciphertext-Policy Attribute-Based Encryption (CP-ABE) where the policy is used for encryption and the attributes satisfying the policy are used for decryption [12].

Many techniques which combine PRE with ABE have been presented in the past [10, 15], however there have been very few studies which discuss the applicability of PRE in the domain of IoT. For e.g., [16] discuss the encryption protocol switching

mechanisms between Identity-Based Encryption (IBE) and ABE, however both operations are too computationally expensive for IoT devices due to the presence of pairing operation. Authors in [17] presented combining ElGamal type Certificate-Based Encryption (CBE) with IBE which may be utilized in the applications involving IoT devices, but IBE does not provide fine-grained sharing of data. To the best of our knowledge, there is no study which provides an abstraction of protocol switching from a CBE-type encryption technique (such as ElGamal) to ABE for IoT applications.

In this paper, we present Light-SABRE (**L**ightweight **S**ticky **A**tribute-**B**ased proxy **R**e-Encryption) for privacy preserving data distribution in IoT environment which utilizes sticky policy approach and combines it with Proxy Re-Encryption between ElGamal and ABE type encryption systems. Through such mechanism, the IoT devices can encrypt the data with a simple CBE encryption (such as ElGamal in this case) and the proxy can convert the CBE cipher into ABE cipher, such that the ABE cipher is derived from the policies attached by the data owner. The data owner attaches those policies to the (ElGamal) encrypted data in the form of sticky policies which are received by the proxy. Thus, the trust from the proxy can be removed and the access control is determined entirely by the sticky policies attached to the encrypted data, thus preserving privacy of the data owners.

The rest of this paper is organized as follows. In Section 2, we present the goal of this research and briefly review the definitions of concepts used in this paper. In Section 3, we present the overall architecture of Light-SABRE. In Section 4, we provide application scenario, followed by discussion in Section 5. We conclude our work in Section 6.

2 Background and Overview

2.1 Related studies

We first present the overall goal of this paper, the problem to solve and an overview of the solution. We aim to solve the privacy challenges faced by the data owner during data storage in cloud and their sharing with third parties. We want to limit our focus to IoT devices because of their ubiquity in diverse application areas which can have significant privacy repercussions due to accidental and purposeful leakage. Many research studies exist in the related area of privacy preserving data storage and distribution concerning to IoT devices and application areas. Li et al [18] proposed privacy preserving data aggregation scheme for MEC assisted IoT devices. In addition to privacy, their scheme provides source authentication and integrity. However, they require presence of MEC for their scheme to work. Yang et al [19] proposed a data storage and distributing mechanism preserving patient's privacy and at the same time providing adaptive access control during emergency situations. They developed a novel approach of smart deduplication over ABE to save storage and transfer cost of encrypted data. However, in their scheme, the data from the IoT devices is first aggregated into a medical file which is then encrypted using ABE. Therefore, the solution requires a trusted encrypting machine between IoT devices and storage service.

Zhang et al [20] improved upon Yang et al proposal and implemented their solution on Intelligent Transportation System. However, they too assumed a presence of encrypting agent at the Data Owner side who can use ABE. To reduce the computational overhead on IoT devices and introduce lightweight solution in privacy-preserving IoT access control, Nasiraei et al [21] proposed combining AES-based symmetric key encryption with ABE. The data owners first encrypt the data with AES, then encrypt the AES key with the proposed ABE algorithm. However, they still assume that IoT devices can perform the asymmetric ABE encryption. Moreover, the data owner must use new keys for each new data. It does not provide solution in cases where data owner has limited computational power to perform pairing operations used in ABE. In addition to low-cost ABE encryption, preserving privacy of the access policy has also been studied in recent years. In this regard, Phuong et al [22] and Sun et al [23] proposed a lightweight and privacy-aware data access control for IoT devices by utilizing short vector transformation of attributes and access policy for CP-ABE and offline pre-computing of encryption variables. The goal is to provide the low-cost fine-grained encryption with added privacy benefits due to hidden policy. However, in many application areas, the IoT devices are rarely offline for long time enough to perform such computations. To reduce the computational burden during encryption and decryption of ABE ciphertext, [24] proposed outsourcing of pairing operation to a semi-trusted server. However, this still requires the data owners to compute multiple exponentiations in the target group for every outsourced pairing, which is not efficient. Therefore, we argue that an encryption switching protocol between a lightweight encryption mechanism at the Data Owner side and more sophisticated encryption mechanism at the data storage side is highly effective for privacy-preserving data storage and access control for IoT applications. In this regard, Green et al [25] proposed outsourcing the decryption of ABE cipher to a semi-trusted cloud which can transform an ABE cipher into an ElGamal style cipher by using a transformation key provided by the user. Similar to our assumption, the cloud cannot read any part of user's message. The authors also proved the CPA and replayable CCA security, but in the random oracle model. While their approach can provide efficient decryption, they do not provide any lightweight mechanism during data transmission when the data owner has limited computation power. Moreover, the transformation key is also generated by the data owner, which may not be possible in case of lightweight devices. They assume that the ABE ciphertext is available a-priori as well as rely on data users to present transformation key which can be inefficient. This work attempts to solve the problem of preserving data privacy during data transmission phase when the data owners do not own a computationally powerful device such as a smart watch or medical sensors etc.

2.1.1 Contribution: We have made a list of concrete problems solved by this work:

1. How to preserve privacy of the data owners in the computationally limited IOT environment.

2. How to communicate the data such that even cloud service providers may not be able to decrypt it.
3. How to achieve fine-grained sharing of data with the authorized third parties.

In this paper we propose that 1. is achieved if Sticky Policies are utilized by cloud service providers for access control. Meanwhile 2. and 3. are achieved by using PRE and ABE respectively. Our main contribution lies in our key generation mechanism which provide encryption parameters to the data owners so they can simply encrypt their private data with ElGamal using one exponentiation. Meanwhile, the key generation mechanism will generate PRE key shared to the CSP by virtue of which the ElGamal ciphers can easily be converted to ABE ciphers. The whole transformation of ElGamal ciphers into ABE ciphers is guided by the sticky policies set by the data owners. In the following subsections, we provide brief overview of these underlying technologies.

2.2 Overview of concepts

2.2.1 Sticky Policies: The Sticky Policies were proposed by [26] to realize fine-grained user control on how their data gets accessed and used. In this paradigm, the security policies (or access control policies) are sent along with the (preferably encrypted) data units and follow them along their whole life cycle. Such policies *should* be tamper-proof [4] and *must* state the requirements or conditions to be applied to the data before its access or use. A trust authority is required to check the compliance of a given agent with the policy, and thereby deciding whether the access should be granted or denied.

Application in Light-SABRE: In our architecture, the data owner (or the IOT device) must first attach a policy to the data before sending to a CSP. We allow a semi-trusted proxy (located in the CSP) to parse and read the policy and apply fine-grained access control compliant with the attached policy. Such access control is implemented through Attribute-Based Encryption (ABE). A schematic representation of data flow in a sticky policy mechanism is shown in Figure 1.

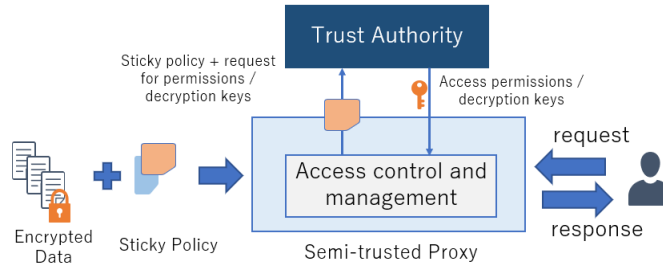


Fig. 1. Schematic representation of data flow in Sticky Policy mechanism.

2.2.2 Attribute Based Encryption: Definition 1: An attribute-based encryption (ABE) usually consists of four algorithms.

1. **ABE.Setup**($\lambda, \mathbb{U}$): intake a security parameter λ and description universe $\mathbb{U}$, output the public parameters PK and a master key MSK. We assume that PK is implicitly seen as input for the following algorithms.
2. **ABE.KeyGen**(MSK, $\mathbb{A}$): intake the master key MSK and a description $\mathbb{A}$, output a private key SK.
3. **ABE.Encrypt**($\mathcal{M}, \mathbb{B}$): intake a message $\mathcal{M}$, and a description $\mathbb{B}$, output a ciphertext CT.
4. **ABE.Decrypt**(CT, SK): intake a ciphertext CT which contains a description $\mathbb{A}$, and a private key SK corresponding to another description $\mathbb{B}$. If $\mathbb{B}$ matches $\mathbb{A}$ the algorithm decrypts the ciphertext and returns the message $\mathcal{M}$; otherwise, return $\perp$.

When $\mathbb{A}$ is a set of attributes over $\mathbb{U}$ and $\mathbb{B}$ is an access policy, the definition is for CP-ABE; if the case is the other way round, that is for KP-ABE.

Application in Light-SABRE: In our architecture, we utilize CP-ABE for its flexibility and its easy alignment with the sticky policy approach, as the attached policies can be directly used for encryption. Only the third parties which possess the attributes satisfying the sticky policies would be able to decrypt the ciphertext, therefore realizing the fine-grained data access. Since the initial data sent by the IoT is encrypted by a much simpler ElGamal encryption technique, it must be converted to ABE cipher for realizing fine-grained access control. In our architecture, we allow the semi-trusted proxy (from here on we will use the terms proxy and CSP interchangeably based on the situation of use) to modify the cipher from ElGamal to ABE by using Proxy Re-Encryption (PRE) technique.

2.2.3 Proxy Re-Encryption (PRE): Definition 1: A Proxy Re-Encryption (PRE) usually consists of four algorithms.

1. **PRE.Setup**($\lambda, \mathbb{U}$): ($ES1.PK, ES1.MSK$) $\leftarrow ES1.Setup(\lambda, \mathbb{U})$ and ($ES2.PK, ES2.MSK$) $\leftarrow ES2.Setup(\lambda, \mathbb{U})$. Set $PK = (ES1.PK, ES2.PK)$ and $MSK = (ES1.MSK, ES2.MSK)$.

Here ES1 and ES2 are Encryption Schemes 1 and 2 respectively. Note that λ is the same security parameter for both the schemes and the $ES1.PK, ES2.PK$ could be held by two distinct trusted parties.

2. **PRE.KeyGen**(MSK): $SK_{\mathbb{A}} \leftarrow \delta.KeyGen(MSK, \mathbb{A})$, where $\delta \in \{ES1, ES2\}$ and $\mathbb{A} \in \{ES1 \text{ description}, ES2 \text{ description}\}$
3. **PRE.Encrypt**($\mathcal{M}, ES1.PK$): $CT_{\mathbb{A}} \leftarrow \delta.Encryption(\mathcal{M}, \mathbb{A})$. We assume that ES1 and ES2 share the same message domain in the definition.
4. **PRE.ReEncrypt**($CT_{\mathbb{A}}, RK_{\mathbb{A} \rightarrow \mathbb{B}}$): intake a ciphertext $CT_{\mathbb{A}}$ under the description $\mathbb{A}$ and a re-encryption key $RK_{\mathbb{A} \rightarrow \mathbb{B}}$, output a re-encrypted ciphertext $CT_{\mathbb{B}}$.
5. **PRE.Decrypt**(CT, SK): $\mathcal{M} / \perp \leftarrow \delta.Decrypt(CT, SK)$.

Note that we assume the above conversion definition between ES1 and ES2 should share the same message domain $\mathcal{M}$ (so that the conversion can be executed smoothly).

The schematic representation of a general PRE mechanism is shown in Figure 2. In our case, ES1 is ElGamal and ES2 is ABE. Therefore, $\mathbb{A}$ becomes public parameters

in ElGamal Encryption Scheme and the Sticky Policy respectively, meanwhile $\mathbb{B}$ is ABE. In our system, we utilize a Key Generation Center (or KGC) trusted by the data owner to generate and manage the encryption and decryption keys for both ElGamal and ABE. By delegating such a task to the KGC, the computational burden on the data owner devices is diminished. The schematic for ElGamal $\rightarrow$ ABE Proxy Re-Encryption is shown in Figure 3.

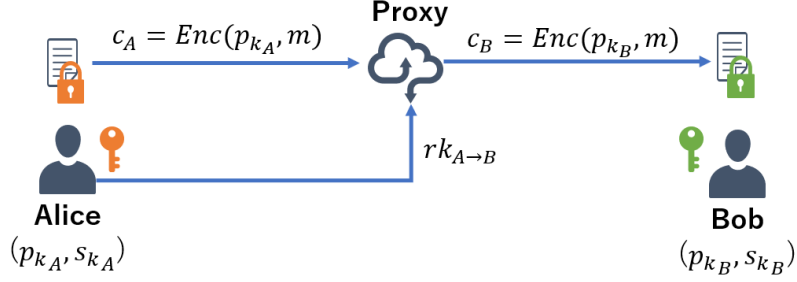


Fig. 2. Schematic representation of Proxy Re-Encryption (PRE); pk, sk are public and secret keys respectively, m is the message, c is the ciphertext and rk is the re-encryption key.

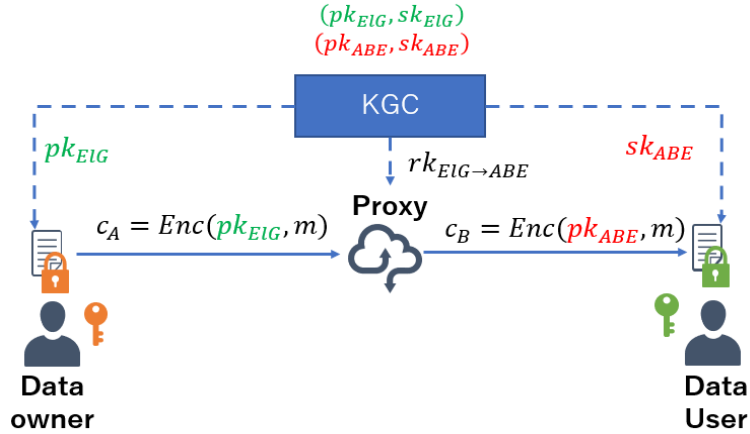


Fig. 3. Schematic of ElGamal (ElGamal) $\rightarrow$ ABE Proxy Re-Encryption mechanism used in our architecture; KGC is responsible for generating and managing the keys.

Application in Light-SABRE: To generate keys for decryption with ElGamal, ABE as well as re-encryption at the proxy (CSP), our scheme utilizes a trusted KGC. The idea is that the KGC can receive the sticky policies from the CSP and generate re-encryption keys. Similarly, it can receive attributes from the third party and generate decryption keys. However, we ensure that KGC never receives any data from the CSP by following an **honest-but-curious** (semi-trusted) assumption on CSP [18, 20]. This assumes that the CSP performs its job honestly by only delivering the policy to the KGC but may itself try to peek into private communications by trying to obtain plaintext from the stored ciphertext. However, thanks to PRE, the CSP can never learn the plaintext information from the stored ciphertext. A schematic of our Light-SABRE

architecture is shown in Figure 4. The data owners own multiple IoT devices which collect sensitive user data. The devices are registered with a CSP for data storage and distribution. Before sharing the data, the data owners can set policies for different categories of data according to their privacy requirements. The IoT devices first encrypt the data (with ElGamal) and attach the set policies to the encrypted data. The data is sent to the CSP for storage, where the CSP parses the sticky policies and send it to the KGC. The KGC analyses the policy and generate re-encryption keys and send them back to the CSP. The CSP then applies the re-encryption algorithm (**PRE. ReEncrypt**) to convert the ElGamal cipher into ABE cipher and stores the re-encrypted data. When a third-party wants to request access to the stored data, the CSP presents the re-encrypted data to the third party. If they wish to access the decryption keys, the third party may present its attributes to the KGC. The KGC generates and sends the decryption keys back to the third party. If the attributes satisfy the policies, the decryption is successful.

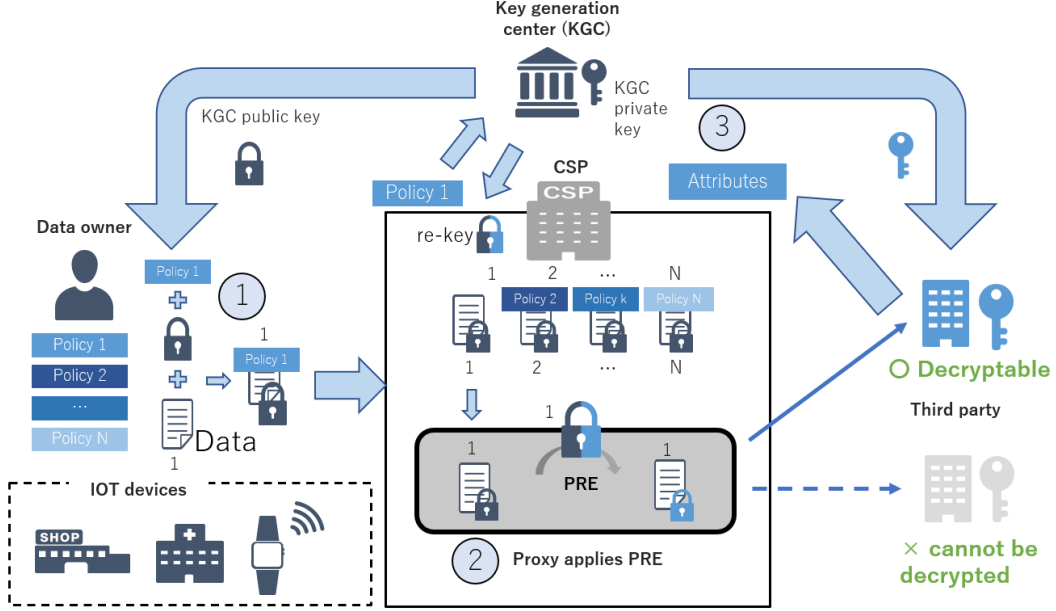


Fig. 4. Schematic of our architecture showing steps (1) Initial encryption and policy attachment; (2) PRE application by the proxy, and (3) ABE Decryption key generation by the KGC.

3 Light-SABRE Architecture

3.1 Workflow of Light-SABRE

The overall workflow of our architecture is described in Figure 5. Our architecture involves interaction between four parties namely data owners (along with IoT devices), Key Generation Center (KGC), Cloud Service Provider (CSP) and third party. (Note

that the proxy is implemented inside the CSP). We divide our workflow in several phases each involving different aspect from data generation to the data access.

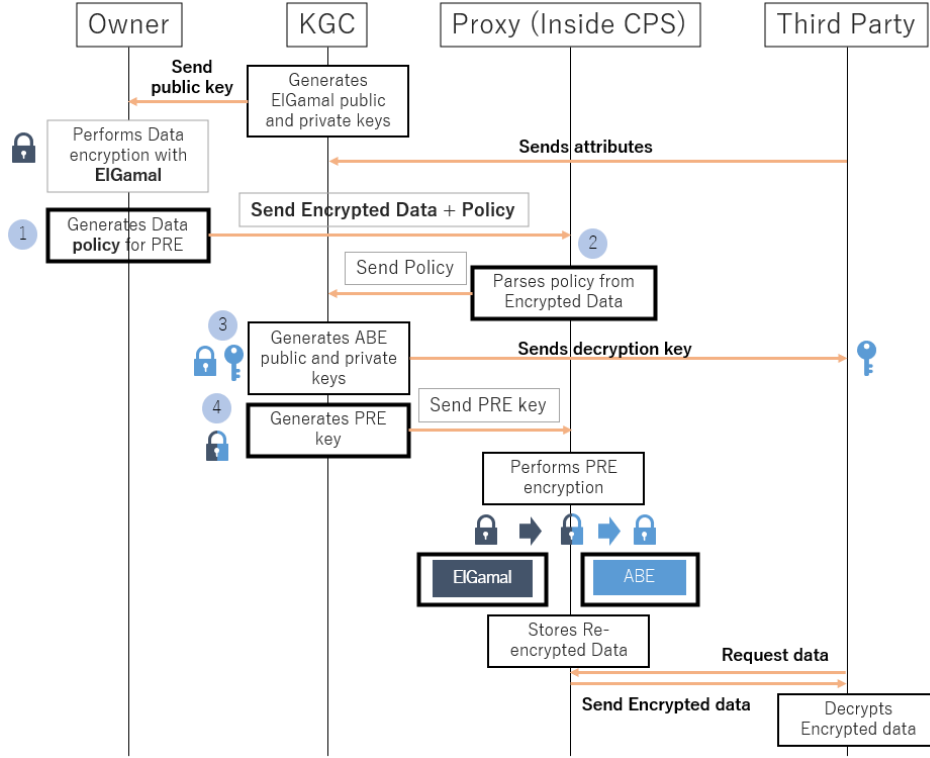


Fig. 5. Detailed operational flow of Light-SABRE. It consists of three phases: (a) Policy creation and Data transfer phase; (b) Policy parsing and Proxy Re-Encryption phase; and (c) Key Generation Phase and decryption.

3.2 Key phases in Light-SABRE workflow

3.2.1 Phase 1: Data transfer from owners to CSP. The IoT devices owned by the users generate diverse data by collecting private user information such as health, finance, etc. Different type of data requires different security and privacy policies, and the data owners are provisioned to generate various policies by stating their privacy and security requirements in the **Policy Creation Module**. The Policy Creation Module is integrated with the Attribute-Based Access Control (ABAC) model which allows data owners to set fine-grained access control requirements to access their data by utilizing attributes. For e.g., data owners can set different requirements for different permissions on the data, such as read, write, execute and modify, by setting different attribute conditions and constraints which must be satisfied for each data operation. For instance, *location* or *role* attribute of the third-party user may determine which operation is allowed on the requested data.

To encrypt the data generated by the IoT, the data owner first selects a trusted Key Generation Center (KGC) and establishes connection with it. After the successful connection is established, the KGC sends ElGamal public key to the data owner. The data owner encrypts each data with the received public key and attaches appropriate policies to the encrypted data. This whole blob of encrypted data and attached policy is sent to the CSP for storage and potential future access from authorized third parties.

All the operations of phase 1 are shown below.

1. **Policy Creation** ($\mathcal{M}$, ABAC): outputs $\mathcal{P}_{\mathcal{M}}$
2. **Trust Establishment** with KGC: KGC sends $ElG.PK$ to the data owner.
3. **ElGamal Encryption**: Data owner applies $ElGamal.Encrypt(\mathcal{M}, ElG.PK) \rightarrow CT_{ElG}$ where CT_{ElG} stands for ElGamal cipher.
4. **Encrypted Blob**: $CT = (CT_{ElG}, \mathcal{P}_{\mathcal{M}})$ is received by the CSP.

3.2.2 Phase 2: Policy parsing and Proxy Re-Encryption (PRE). After receiving the encrypted blob CT , the proxy inside the CSP parses the policy from the CT with the help of Policy Parsing Module. The first function of the proxy is to send the policy to the KGC for PRE key generation. When the proxy receives the re-encryption keys from the KGC, it immediately starts applying PRE to the encrypted data CT_{ElG} . Since the data is encrypted when the proxy receives it, the proxy never learns any plaintext information from the CT_{ElG} . Later, when the proxy receives data access request from third party, it can send the re-encrypted data to the third party.

All the operations of phase 2 are shown below.

1. **Policy Parsing** (CT): outputs CT_{ElG} and $\mathcal{P}_{\mathcal{M}}$. Send $\mathcal{P}_{\mathcal{M}}$ to the KGC.
2. **Receive PRE-key**: Proxy receives the re-encryption key $RK_{ElG \rightarrow ABE}$ from the KGC.
3. **PRE**: Proxy performs PRE (CT_{ElG} , $RK_{ElG \rightarrow ABE}$): outputs CT_{ABE} .
4. **Access request**: Proxy receives data access request for CT_{ABE} from third party.
5. **Access response**: Proxy sends CT_{ABE} to the third party (after optionally performing access control check).

3.2.3 Phase 3: Key Generation Phase (and decryption). The trusted KGC is an all-powerful server which can generate decryption keys for every encrypted data in the system. Therefore, we ensure that the KGC never receives the encrypted data. We ensure that by assuming that the proxy is an honest-but-curious proxy. Therefore, it will only share the policy with the KGC without sharing the encrypted data. KGC utilizes its own private key and the received policy $\mathcal{P}_{\mathcal{M}}$ to generate re-encryption key for the data. Additionally, on receiving the attributes S' from the third party, it can create ABE decryption keys corresponding to the attributes.

All the operations of phase 3 are shown below.

1. **KGC PRE-key generation** ($KGC.MSK, ElG.SK$, $\mathcal{P}_{\mathcal{M}}$): outputs re-encryption keys $RK_{ElG \rightarrow ABE}$. Send $RK_{ElG \rightarrow ABE}$ to the proxy.

2. **KGC decryption key generation** ($KGC.MSK, \mathcal{P}_M, S'$): outputs ABE decryption keys SK_{ABE} . Send SK_{ABE} to the third party. Optionally verify the authenticity and integrity of the attributes.

Once the third party receives the decryption keys, it can decrypt the encrypted data if the attributes satisfy the policies set by the data owner, as the encrypted data and sticky policies are mathematically entangled.

3.3 Construction of our scheme

We build our ElGamal-ABE PRE scheme based on the efficient Waters-ABE scheme [27] which proceeds as follows:

KGC $\Rightarrow$ Setup: Let G, G_T be bilinear groups of prime order p , such that there exists a pairing $e: G \times G \rightarrow G_T$ between G, G_T . In addition, select random elements $\alpha, \beta, \gamma \in Z_p^*$ and a generator $g \in G$. The KGC sets the public parameters as $pp \rightarrow (g, e(g, g)^\alpha, g^{\alpha\beta})$ and the master secret key as $KGC.MSK \rightarrow (\alpha, \beta, g^\alpha, g^\beta)$. The KGC sets the parameters for ElGamal encryption as follows:

$$(ElG.PK, ElG.SK) = (g^{\alpha\gamma}, \gamma) \quad (1)$$

Data Owner $\rightarrow$ Encrypt (M, pp, pk_{ElG}): The data owner (IoT device) chooses a random exponent $x \in Z_p^*$ and generates the ElGamal ciphertext as follows:

$$CT_{ElG} = (C_{1_{ElG}}, C_{2_{ElG}}, C_{3_{ElG}}) = (M \otimes (e(g, g)^\alpha)^x, g^x, g^{\alpha\gamma x}) \quad (2)$$

Data Owner $\Rightarrow$ Policy Generation: Our scheme expresses the ABAC policy $\mathcal{P}_M$ using a Linear Secret Sharing Scheme (LSSS) access structure represented by matrix $(\mathbb{M} (m \times n), \rho)$ similar to Waters scheme [27], where ρ associates the rows of the matrix $\mathbb{M}$ with attributes $a \in S$. The payload is set as $\wp : CT_{ElG} || \mathcal{P}_M$.

CSP $\Rightarrow$ Policy Parsing: The CSP accepts the payload $\wp$ and extracts the components $(\mathcal{P}_M, C_{2_{ElG}}, C_{3_{ElG}})$ and send them to the KGC.

KGC $\Rightarrow$ Re-Encryption Key Generation: KGC accepts the policy as $(\mathcal{P}_M, C_{2_{ElG}}, C_{3_{ElG}})$. It first chooses three random elements; $r \in Z_p^*$, which is a chosen randomness for each message, $s \in Z_p^*$, which is a chosen secret for each message and $r' \in Z_p^*$, which is a chosen randomness for each access request from the CSP. Then for the ABAC policy matrix $\mathbb{M}$, it first creates a vector $\vec{v} = (s, y_1, y_2, \dots, y_n)$ and set $\sum_i \lambda_i w_i = s$, where $\lambda_i = \vec{v} \cdot \mathbb{M}_i$ where $\mathbb{M}_i$ is the vector corresponding to the i^{th} row of $\mathbb{M}$. The KGC chooses random $h_1, h_2, \dots, h_U \in G$ where U is the total number of attributes in the system. Then it generates random elements $r_i \in Z_p^*$ for each attribute in S . It then sets,

$$r'' = r' + r \quad (3)$$

The re-encryption key is calculated as follows:

$$RK_{ElG \rightarrow ABE}: (rk_0, rk_1) = \left((g^s)^{\frac{1}{r}}, g^{r''\beta} \right) \quad (4)$$

The ABE cipher is calculated as follows:

$$C = (g^s, \{g^{\beta\lambda_i} h_{\rho(i)}^{-r_i}\}, \{g^{r_i}\}) \quad (5)$$

CSP $\Rightarrow$ ElGamal $\rightarrow$ ABE Re-Encryption: Using inputs as CT , $RK_{\text{ElG} \rightarrow \text{ABE}}$ and ABE cipher C , the CSP computes the ciphertext CT_{ABE} as:

$$C'_1 = C_{1_{\text{ElG}}} \otimes e(g^{\alpha\gamma}, rk_0) \otimes e(C_{2_{\text{ElG}}}, rk_1); C'_2 = g^s \times g^x; C'_3 = \{g^{\beta\lambda_i} h_{\rho(i)}^{-r_i}\}; C'_4 = \{g^{r_i}\} \quad (6)$$

KGC $\Rightarrow$ Decryption Key generation: With inputs as attribute set S' received from third party, and the master secret key $\text{KGC.MSK} = (\alpha, \beta)$, the KGC first selects random $h_1, h_2, \dots, h_{S'} \in G$ from the set $\{h_1, \dots, h_U\}$. The decryption key is computed as follows:

$$\text{SK}_{\text{ABE}} = (d_0, d_1, d_2, d_3) = (g^{\alpha+r\beta}, g^{r'\beta}, g^r, g^{r'}, h_x^r, h_x^{r'} \forall x \in S') \quad (7)$$

Third party $\Rightarrow$ Decryption: With inputs as CT_{ABE} and SK_{ABE} , calculate:

$$d = \frac{e(C'_2, d_0 \times d_1)}{\prod_i (e(C'_3, d_2 \times d_3) \times e(C'_4, d_3 \times d_4))^{w_i}}; M = \frac{C'_1}{d} \quad (8)$$

If $S = S'$, the above equation will reveal the message M , else it will return $\perp$.

3.4 Correctness

Decryption of ElGamal cipher by KGC: With using ElG.SK , it is possible for the KGC to decrypt the ElGamal cipher CT_{ElG} by computing:

$$M = \frac{C_{1_{\text{ElG}}}}{e\left(\frac{1}{C_{3_{\text{ElG}}}} g, g\right)} \quad (9)$$

Decryption of ABE cipher by third-party: Assuming $S = S'$, let's first compute $\prod_i (e(C'_3, g^r \times g^{r'}) \times e(C'_4, h_x^r \times h_x^{r'}))^{w_i}$.

$$e(C'_3, g^r \times g^{r'}) \times e(C'_4, h_x^r \times h_x^{r'}) = e(g^{\beta\lambda_i} h_{\rho(i)}^{-r_i}, g^{r''}) \times e(g^{r_i}, h_{\rho(i)}^{r''}) = e(g, g)^{\beta r'' \lambda_i} \quad (10)$$

$$\prod_i (e(C'_3, g^{r''}) \times e(C'_4, h_x^{r''}))^{w_i} = \prod_i e(g, g)^{\beta r'' \lambda_i w_i} = e(g, g)^{\beta r'' \sum_i \lambda_i w_i} = e(g, g)^{\beta r'' s} \quad (11)$$

Now:

$$(C'_2, d_0 \times d_2) = e(g^s \times g^x, g^{\alpha+r\beta} \times g^{r'\beta}) = e(g, g)^{(s+x)(\alpha+(r+r')\beta)} = e(g, g)^{(s+x)(\alpha+r''\beta)} \quad (12)$$

Therefore:

$$d = \frac{e(C'_2, d_0 \times d_2)}{\prod_i (e(C'_3, d_1 \times d_2) \times e(C'_4, d_3 \times d_4))^{w_i}} = \frac{e(g, g)^{(s+x)(\alpha+r''\beta)}}{e(g, g)^{\beta r'' s}} = e(g, g)^{\alpha s + \alpha x + r'' \beta x} \quad (13)$$

Finally:

$$C'_1 = C_1 \times e(rk_0, pp_{ElG}) \times e(rk_1, C_{2_{ElG}}) = M \times e(g, g)^{\alpha x} \times e(g^{s/\gamma}, g^{\alpha\gamma}) \times e(g^{r''\beta}, g^x) = M \times e(g, g)^{\alpha x + \alpha s + r''\beta x} = M \times d \quad (14)$$

3.5 Security Analysis

3.5.1 CPA Security Game: We show that our scheme is IND-CPA secure in the Decisional q-parallel Bilinear Diffie Hellman Exponent (BDHE) Assumption.

Theorem 3.1: Suppose the Waters CP-ABE scheme is IND-CPA secure, then our Light SABRE scheme is also IND-CPA secure PRE scheme.

The following game describes the selective security of the underlying scheme even if the adversary receives the re-encryption key which can transform the ElGamal ciphertext into ABE ciphertext.

Init: The adversary chooses a target policy $\mathcal{P}^*$ and sends it to the challenger.

Setup: The challenger runs the setup algorithm and distributes the public keys to the adversary

Phase 1: The adversary can make the following queries:

1. **Extract_rekey($\mathcal{P}$):** Adversary can repeatedly make queries for re-encryption key for any policy $\mathcal{P}$ of their choice.

2. **Extract_secret(S):** Adversary can repeatedly make queries for ABE private key for any set of attributes S such that they do not satisfy the policy $\mathcal{P}^*$

Challenge: The adversary submits equal length messages M_0 and M_1 . The challenger flips a coin b and encrypts M_b under $\mathcal{P}^*$. The challenge ciphertext is CT^* .

Phase 2: Same as Phase 1.

Guess: A outputs b' , if $b' = b$, the Adversary wins.

3.5.2 Security Proof: We attempt to prove this by contradiction. We assume an adversary $\mathcal{A}$ which has non negligible advantage $Adv_{\mathcal{A}}$ in the selective security game against our construction. We show how to build a simulator $\mathcal{B}$ that can play the Waters CP-ABE game. For the sake of simplicity, we simply refer to the Waters proof of their efficient CP-ABE scheme. If we can build a simulator $\mathcal{B}$ that can play the Waters game, our proof is complete.

The simulator $\mathcal{B}$ maintains a table T which would contain the public parameters PK, the policy $\mathcal{P}$, the attributes S which satisfy the policy $\mathcal{P}$, the ElGamal-ABE PRE key RK and the secret key SK.

Setup: The simulator $\mathcal{B}$ creates the public parameters PK corresponding to both ElGamal and ABE and send them to the adversary $\mathcal{A}$. $PK = (g, e(g, g)^\alpha, g^{\alpha\beta})$.

Phase 1: $\mathcal{B}$ initializes an empty table T and an integer $j = 0$. It answers the adversary queries as follows:

Extract_rekey($\mathcal{P}$): $\mathcal{B}$ sets $j = j + 1$. $\mathcal{A}$ submits the set of attributes S' . It proceeds in two ways:

1. If S does not satisfy $\mathcal{P}^*$, $\mathcal{B}$ runs the KGC PRE key generation and KGC decryption key generation and outputs (RK', SK') respectively.
2. If S satisfies $\mathcal{P}^*$, $\mathcal{B}$ chooses a fake PRE key by running KGC PRE key generation in the following manner: $\mathcal{B}$ chooses random integers s_1, r_1, z_1, z_2 and set $RK_{ElG \rightarrow ABE}: (rk_0, rk_1) = ((g^{s_1})^{z_1}, g^{r_1 z_2})$. It then chooses random

integers s_2, β' , the set of integers $k_1, k_2, \dots, k_i$, another set of integers $l_1, l_2, \dots, l_i$ and set $C = (g^{s_2}, \{g^{\beta' k_i} h_{\rho(i)}^{-l_i}\}, \{g^{k_i}\})$
 $\mathcal{B}$ stores $(j, \mathcal{P}, S, \text{RK}, \text{SK})$ in $\mathcal{T}$ and returns RK to $\mathcal{A}$.

Extract_secret(S): $\mathcal{A}$ can ask for private keys corresponding to any set of attributes, such that S does not satisfy $\mathcal{P}^*$. $\mathcal{B}$ checks if there is an entry corresponding to S in table $\mathcal{T}$. If the entry exists, $\mathcal{B}$ returns SK to $\mathcal{A}$ else returns $\perp$.

Challenge: $\mathcal{A}$ submits equal length messages M_0 and M_1 . $\mathcal{B}$ forwards them to the challenger and obtains CT^* . Then $\mathcal{B}$ sends CT^* to $\mathcal{A}$ as the challenge ciphertext.

Phase 2: $\mathcal{B}$ continues to answer the **Extract_rekey** and **Extract_secret** queries from $\mathcal{A}$, except that the set of attributes submitted by $\mathcal{A}$; S' does not satisfy $\mathcal{P}^*$.

Guess: $\mathcal{A}$ outputs guess b' of b . Then $\mathcal{B}$ outputs b' .

As the output RK is evenly distributed in G , the fake PRE key is computationally indistinguishable from the real PRE-key, $\mathcal{A}$ can only distinguish the fake PRE key from the real one with a negligible probability ϵ . Thus, except this negligible probability, $\mathcal{B}$ has perfectly simulated the Waters CP-ABE game. So, if $\mathcal{A}$ has a non-negligible advantage against our construction scheme, then according to our simulation:

$$\Pr(\mathcal{A}(b' = b)) = \Pr(\mathcal{B}(b' = b)) + \epsilon \quad (15)$$

$$\text{i.e.,} \quad \Pr(\mathcal{B}(b' = b)) = \frac{1}{2} + \text{Adv}_{\mathcal{A}} - \epsilon \quad (16)$$

Hence, $\mathcal{B}$ can attack the Waters CP-ABE Waters with a non-negligible advantage.

3.6 Performance Analysis

Scheme	Initial Encryption	KGC Setup	PRE-Key Generation	ABE Key Generation	PRE-Encryption	ABE Decryption
Phuong et al [22]	$(12l + 38)E_G + E_{G_T}$	$(8l + 27)E_{G_0} + E_{G_T}$	$\times$	$13(n + 3)E_G$	$\times$	$(4l + 14)P$
Sun et al [23]	$(2l + 6)E_G + E_{G_T}$	$(l + 5)E_G + P$	$\times$	$(l + 6)E_G$	$\times$	$3P + (l + 3)E_G$
Green et al [25]	$M_{G_T} + P + (2 + 2l) \times E_G$	$2E_G + P$	$(2 + l) \times E_G$	$(2 + l) \times E_G$	$(2 + l)P$	$2E_{G_T}$
Our Scheme	$2E_G + M_{G_T} + E_{G_T}$	$4E_G + P$	$(3 + 2l) \times E_G$	$(4 + 2l) \times E_G$	$2P + M_{G_T} + (2 + 2l) \times E_G$	$(1 + l) \times P$

Table 1. Performance comparison between different lightweight ABE schemes in terms of computational cost. E_G and E_{G_T} refers to exponentiation in source group and target group respectively, M_{G_T} refers to the multiplication in target group and P refers to the pairing operation. " $\times$ " means not applicable.

The performance analysis of our scheme and comparison with related works is presented in Table 1. We show that the initial encryption cost is the lowest for our scheme and therefore most suitable for low powered IoT devices. Although, we note that our scheme is not optimal in terms of decryption cost compared to Sun et al [23],

but it covers much broader policies compared to them (Sun et al does not cover LSSS policy but only the logical AND policies with addition operation).

4 SCENARIO

We describe the practical working of our architecture with the help of a scenario from healthcare fitness application. Assume a data owner Bob owns a smartwatch from company X which monitors and records a multitude of activities: (a) step count; (b) heart rate; and (c) sleep patterns. Assume that the same company X also function as a Cloud Service Provider (CSP) for storing this data. As a data owner, Bob is concerned with the privacy of the data recorded by its smartwatch. According to Bob, different data have different sensitivity levels. While he is happy sharing his step count with friends, he does not want anyone to access his heart rate except his doctor Alice working in Hospital Y. On the other hand, Bob allows a sleep analysis company Z to access his sleep patterns. He doesn't trust the CSP X and is afraid that X may analyze, leak or sell his data to unauthorized parties. In such a scenario, our architecture can help in secure storage and distribution of data only to the authorized parties. By following the ABAC model, the natural language policies assigned by Bob can be converted to the appropriate sticky policies in the Policy Creation Module as shown below.

1. **Step-count:** {Condition: $(User == Friend \cap \overline{Close}) \wedge (Resource == \text{steptotal}) \wedge (Operation == View)$, Action = Allow, Default Action: Deny}
 {Condition: $(User == Friend \cap Close) \wedge (Resource == \text{steptotal} \vee \text{location}) \wedge (Operation == View)$, Action = Allow, Default Action: Deny}
2. **Heart rate:** {Condition: $(User == Alice) \wedge (Resource == \text{heart rate}) \wedge (Operation == View \wedge \text{Write} \wedge \text{Execute})$, Action = Allow, Default Action: Deny}
 {Condition: $(User \text{ Role} == Nurse) \wedge (Hospital = Y) \wedge (Resource == \text{heart rate}) \wedge (Time < 24\text{hrs}) \wedge (Operation == View)$, Action = Allow, Default action: Deny}
3. **Sleep:** {Condition: $(User == Y) \wedge (Resource == \text{sleep}) \wedge (Operation == View \wedge \text{Write} \wedge \text{Execute})$, Action = Allow, Default Action: Deny}

The Light SABRE architecture allows Bob to register himself with any KGC of his choice. The KGC can generate ElGamal, ABE and PRE keys for securely encrypting the data of Bob. The registration process involves KGC to share its certificate to Bob. Bob can trust KGC based on its certificate. Once the registration is completed, KGC can share the public key information for initial ElGamal encryption and the Bob's device can encrypt each data with received public key from the KGC along with attaching the sticky policies associated with them. The rest of the process follows as described in the architecture section. The KGC can only generate the correct decryption keys when it receives the correct attribute set. For instance, while accessing Heart rate

of Bob: A Nurse in Hospital Y shall receive the decryption key which can only decrypt the READ operation, and nothing else. Similarly, the keys provided to any Nurse from Hospital which is not Y, shall not be able to decrypt the encrypted heart-rate data. Thus, even in case of X leaking data to the third-party hospitals, the data is still encrypted and inaccessible.

5 DISCUSSION

The described architecture is one of a kind to provide both privacy to the data owners as well as fine-grained control over who can access which data as authorized by the sticky policies defined by them. The sticky policies strictly govern the access control over the data by means of ABE encryption. The proxy cleverly re-encrypts the data with the encryption as defined by the policy, however the proxy itself cannot decrypt the data. The KGC plays a central role in facilitating the whole process by generating and distributing keys to the desired parties. It should be noted that the security of the whole setup depends on the security of each process, i.e., initial ElGamal encryption, ABE, and Proxy Re-Encryption, respectively. Since we used standard ElGamal and ABE, which are independently proven secure [27, 28], the security of the whole model relies on the security of the proxy re-encryption step which involves switching the encryption protocol by converting the ElGamal cipher into ABE cipher. In the security analysis we showed that our PRE scheme is CPA secure. Moreover, we assume that the KGC is fully trusted and secured, i.e., it cannot leak its secret keys on purpose, or it cannot be attacked to reveal its secret key. This can be ensured by placing KGC in highly trusted computing environment such as secure enclaves [29]. However, the over-reliance on the centralized KGC as proposed in this work can be seen as a weakness. There is nothing preventing KGC to misuse its position, such as distributing incorrect decryption keys, or storing and re-distributing the previous decryption keys to malicious actors. To reduce such problems, a distributed KGC approach can be used where the KGCs compete to win the trust of the users using a rating system, for instance. The distributed nature of KGCs can also prevent DDOS attacks. Verifying whether the correct policy was sent by the CSP and used by the KGC is another challenge. The future work will primarily focus on this limitation. Another issue is regarding the computational cost and time involved in re-encryption of ElGamal ciphertext data by the proxy and decryption of the data by the authorized third-party. If the data is large, it is common knowledge that the asymmetric encryption and decryption consumes significantly more time, compared to if a symmetric encryption-decryption algorithm was used. Therefore, an alternate way to approach the setting is to utilize our algorithms in the key-space instead of message space. In such a setting, the entire flow of Light-SABRE can operate with symmetric keys. This reduces the computational cost and saves time as the data encryption and decryption only occurs with the symmetric key. Note that to protect sensitive operations such as write, and execute, each operation must be encrypted separately. That means, even if the symmetric key was to be used, each key shall be encrypted separately for each operation corresponding to the sticky policy determined by the data owner. For improving the

secure operation of Light-SABRE it is optional but recommended to install an access control mechanism at the CSP. This is to ensure that the unauthorized third-party may not provide polluted attributes to the KGC for obtaining the decryption keys to the data. The CSP may check the validity and integrity of the attributes, their source and timestamp etc. so that multiple parties may not collude together, or the attributes used in the past may not be used again. Then it may send certified attribute-set back to the third parties, who may only use these attributes to obtain decryption key from the KGC. The KGC may not accept any attribute-set not certified by the CSP. To implement such an access control model, the CSP would need to run at a higher privilege server for certifying the attribute-set of the third parties. Such comprehensive system may be presented as a future work. In this work we only proposed the theoretical advantage, and it currently lacks the evaluation of performance and scalability, which limits its immediate applicability. A full evaluation on the actual test bed will be presented as a part of future work.

6 CONCLUSION

In this study, we introduced Light-SABRE, a privacy preserving solution for data storage and distribution in IoT environment. The objective of this study was to implement a solution to the privacy problem faced by data owners in storage and distribution of data in external cloud, to provide them a higher degree of data control and allow them to set access control policies on sensitive data shared by their devices to the proprietary storage servers. Our solution achieves this by combining Sticky Policy approach and combining it with Attribute-Based Encryption (ABE), through Proxy Re-Encryption (PRE) technique. The data is initially encrypted by using a lightweight public key encryption (ElGamal) technique, which is converted into a more sophisticated ABE cipher by the proxy with the help of PRE compliant to the policy attached by the data owner. The ABE cipher provides flexibility in terms of fine-grained access control. Thus, our architecture provides data privacy while ensuring fine-grained data access control. Our solution is highly efficient for IoT devices as the heavy computation is delegated to the KGC and the CSP, while the IoT devices only perform initial encryption with lightweight ElGamal. Our described solution is CPA secure in q -parallel BDHE assumption. A future challenge is to prove much stronger security such as CCA or prove the security with weaker assumptions. Another challenge exists in that the KGC is fully trusted, and thus the data owner cannot verify whether the sent policy is used for key generation or not. Moreover, if the CSP sent the correct policy to the KGC is impossible to verify in this architecture. Though the lack of performance evaluation limits its immediate applicability, we believe that in the present age of pervasiveness being a norm, our solution can be one of the key enablers to achieve data privacy and increase the control of data owners over their private data stored in and shared by external cloud.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

- [1] Weber RH.: Internet of Things–New security and privacy challenges. *Computer law & security review* 26(1), 23-30 (2010)
- [2] Sicari S, Rizzardi A, Grieco LA, Coen-Porisini A.: Security, privacy and trust in Internet of Things: The road ahead. *Computer networks* 76, 146-64 (2015)
- [3] Roman R, Zhou J, Lopez J.: On the features and challenges of security and privacy in distributed internet of things. *Computer networks* 57(10), 2266-79 (2013)
- [4] Miorandi D, Rizzardi A, Sicari S, Coen-Porisini A.: Sticky policies: A survey. *IEEE Transactions on Knowledge and Data Engineering* 32(12), 2481-99 (2019)
- [5] Nabeel M, Bertino E.: Privacy-Preserving Fine-Grained Access Control in Public Clouds. *IEEE Data Eng. Bull* 35(4), 21-30 (2012)
- [6] Karjoth G, Schunter M, Waidner M.: Privacy-enabled services for enterprises. In: *Proceedings. 13th International Workshop on Database and Expert Systems Applications*, pp. 483-487. IEEE. (2002)
- [7] Di Cerbo F, Trabelsi S, Steingruber T, Dodero G, Bezzi M.: Sticky policies for mobile devices. In: *Proceedings of the 18th ACM symposium on Access control models and technologies*, pp. 257-260. ACM (2013)
- [8] Sicari S, Rizzardi A, Dini G, Perazzo P, La Manna M, Coen-Porisini A.: Attribute-based encryption and sticky policies for data access control in a smart home scenario: a comparison on networked smart object middleware. *International Journal of Information Security* 20, 695-713 (2021)
- [9] Green M, Ateniese G.: Identity-based proxy re-encryption. In: *5th International Conference on Applied Cryptography and Network Security (ACNS)*, pp. 288-306. Springer Berlin Heidelberg, Zhuhai, China (2007)
- [10] Liang K, Fang L, Susilo W, Wong DS.: A ciphertext-policy attribute-based proxy re-encryption with chosen-ciphertext security. In: *5th international conference on intelligent networking and collaborative systems*, pp. 552-559. IEEE, (2013)
- [11] Guo C, Zhuang R, Jie Y, Ren Y, Wu T, Choo KK.: Fine-grained database field search using attribute-based encryption for e-healthcare clouds. *Journal of medical systems* 40, 1-8 (2016)
- [12] Bethencourt J, Sahai A, Waters B.: Ciphertext-policy attribute-based encryption. In: *IEEE symposium on security and privacy (SP'07)*, pp. 321-334. IEEE, (2007)
- [13] Zhang Y, Deng RH, Xu S, Sun J, Li Q, Zheng D.: Attribute-based encryption for cloud computing access control: A survey. *ACM Computing Surveys (CSUR)* 53(4), 1-41 (2020)
- [14] Goyal V, Pandey O, Sahai A, Waters B.: Attribute-based encryption for fine-grained access control of encrypted data. In: *Proceedings of the 13th ACM conference on Computer and communications security*, pp. 89-98. ACM, (2006)
- [15] Liang K, Au MH, Liu JK, Susilo W, Wong DS, Yang G, Yu Y, Yang A.: A secure and efficient ciphertext-policy attribute-based proxy re-encryption for cloud data sharing. *Future Generation Computer Systems* 52, 95-108 (2015)
- [16] He K, Mao Y, Ning J, Liang K, Huang X, Panaousis E, Loukas G.: A new encrypted data switching protocol: bridging IBE and ABE without loss of data confidentiality. *IEEE Access* 7, 50658-68 (2019)
- [17] Matsuo T.: Proxy re-encryption systems for identity-based encryption. In: *First International Conference on Pairing-Based Cryptography–Pairing*, pp. 247-267. Springer Berlin Heidelberg, Tokyo, Japan (2007)
- [18] Li X, Liu S, Wu F, Kumari S, Rodrigues JJ.: Privacy preserving data aggregation scheme for mobile edge computing assisted IoT applications. *IEEE Internet of Things Journal* 6(3), 4755-63 (2018)
- [19] Yang Y, Zheng X, Guo W, Liu X, Chang V.: Privacy-preserving smart IoT-based healthcare big data storage and self-adaptive access control system. *Information Sciences* 479, 567-92 (2019)

- [20] Zhang L, Wang J, Mu Y.: Privacy-preserving flexible access control for encrypted data in Internet of Things. *IEEE Internet of Things Journal* 8(19), 14731-45 (2021)
- [21] Nasirae H, Ashouri-Talouki M.: Privacy-preserving distributed data access control for CloudIoT. *IEEE Transactions on Dependable and Secure Computing* 19(4), 2476-87 (2021)
- [22] Phuong TV, Yang G, Susilo W.: Hidden ciphertext policy attribute-based encryption under standard assumptions. *IEEE transactions on information forensics and security* 11(1):35-45 (2015)
- [23] Sun J, Xiong H, Liu X, Zhang Y, Nie X, Deng RH.: Lightweight and privacy-aware fine-grained access control for IoT-oriented smart health. *IEEE Internet of Things Journal* 7(7), 6566-75 (2020)
- [24] Chevallier-Mames B, Coron JS, McCullagh N, Naccache D, Scott M.: Secure delegation of elliptic-curve pairing. In: 9th IFIP WG 8.8/11.2 International Conference on Smart Card Research and Advanced Application, pp. 24-35. Springer Berlin Heidelberg, Passau, Germany (2010)
- [25] Green M, Hohenberger S, Waters B.: Outsourcing the decryption of {ABE} ciphertexts. In: 20th USENIX security symposium, pp. 523-538. USENIX Security, San Francisco, CA (2011)
- [26] Pearson S, Casassa-Mont M.: Sticky policies: An approach for managing privacy across multiple parties. *Computer* 44(9), 60-8 (2011)
- [27] Bernhard D, Fischlin M, Warinschi B.: On the hardness of proving CCA-security of signed ElGamal. In: 19th IACR International Conference on Practice and Theory in Public-Key Cryptography (PKC). pp. 47-69. Springer Berlin Heidelberg, Taipei, Taiwan (2016)
- [28] Waters B.: Ciphertext-policy attribute-based encryption: An expressive, efficient, and provably secure realization. In: International workshop on public key cryptography, pp. 53-70. Springer Berlin Heidelberg, Taormina, Italy (2011)
- [29] Brandão A, Resende JS, Martins R.: Hardening cryptographic operations through the use of secure enclaves. *Computers & Security* 108, 102327 (2021)